FETCH.AI FETCH-A-THON 3.0 // COMPLETE MASTER ARCHITECTURE & JUDGE VIVA BIBLE

PatchBounty (@aegis) — Universal Multi-Language Autonomous Vulnerability Verification, Live Bargaining & Zero-Leak Dorado Escrow Studio

Team Credits: Built by **Atul Anand, Jeeya, and Vanshika**
Active 6-uAgent Swarm: @aegis • @recon • @specter • @forge (@blade / @kestrel / @titan) • @warden • @vault
Core Protocols & Network: Fetch.ai uAgents + AgentChatProtocol v0.3.0 + AgentPaymentProtocol v0.1.0 + ASI:One 50/50 Dual Brain (asi1 + asi1-mini) + Fetch Dorado Testnet (dorado-1, 0.05 TESTFET)

Section	Complete Coverage Inside This Handbook
Part 1	Why PatchBounty Needs to Exist: Real-World Bug Bounty Economics, Unpaid Patch Theft Problem & Market Comparison Table
Part 2	End-to-End 5-Page Architecture: Zero-Bias Scanning, Clean Code $0 Verdict, Blue Blurred HTTP-402 Vault Lock & Mandatory Sandbox Gate
Part 3	Every Agent & Every AI Model Explained: Ports 8011-8016, Protocols, ASI:One 50/50 Load Balancer & AST/Subprocess Engines
Part 4	Live Data Flows: GitHub API, Live US NIST NVD CVE API, Session Persistence, & Public Fetch.ai Block Explorer Verification URLs
Part 5	45 Real Hackathon Judge Viva Questions & Word-for-Word Winning Answers (Both Technical English + Quick Hinglish Speaking Points!)

PART 1 — WHY PATCHBOUNTY EXISTS: PROBLEM STATEMENT & MARKET ECONOMICS

1. The Broken Economics & Two-Sided Trust Failure in Traditional Bug Bounties

Every modern software company — from fintech payment startups and SaaS platforms to Web3 DeFi smart contracts — faces severe exposure to exploitable code vulnerabilities such as SQL Injection (CWE-89), OS Command Execution (CWE-78), Unsafe Deserialization (CWE-502), Path Traversal (CWE-22), and Smart-Contract Reentrancy (CWE-841). Yet the global bug-bounty ecosystem (HackerOne, Bugcrowd, Synack, Immunefi) suffers from three deep structural failures that lock out 95% of small and mid-sized engineering teams:

1. Unaffordable Cost & Slow Turnaround: On traditional human platforms, a single Critical SQL Injection or Remote Code Execution report costs an average of $2,400 to $6,200 USD (plus 20% to 27% platform middleman commission) and takes 7 to 21 days of manual triage. Even after paying thousands of dollars, the company receives ONLY a vulnerability report — its own engineers must still spend days writing, testing, and regression-checking the actual code patch!

2. The Researcher's Theft Dilemma (Unpaid Self-Patching): In real-world bug bounties, if a security researcher reveals the exact file path, exact line number, and fixed code BEFORE payment settles, dishonest or cash-strapped businesses simply fix those two lines themselves in GitHub and reject or ghost the researcher's payout ('duplicate / internal fix').

3. The Company's Blind-Payment Dilemma (Paying for Broken Fixes): Conversely, if a business is asked to pay cryptocurrency or cash BEFORE seeing the fix, how does the business know the researcher found real bugs (rather than running a fake alert on clean bug-free code) OR that the proposed fix actually compiles, blocks the exploit, and passes unit tests?

PatchBounty (@aegis) resolves ALL THREE failures simultaneously through a 6-Agent Fetch.ai uAgents architecture combining: (a) Multi-Language Top-to-Bottom Code Inspection where clean code honestly returns 0 bugs & $0 cost, (b) Blind Cryptographic Sandbox Breach Proofs + Blue Frosted-Blur Vault Sealing (HTTP 402 server-side redaction before payment), (c) Real-World Live Industry Market Benchmark vs Competitive AI Agent Bargaining (saving ~88% to 92% vs human platforms with zero upfront budget peeking), (d) Mandatory Pre-Payment @warden Sandbox Verification Gate, and (e) Real Fetch.ai Dorado Testnet (dorado-1) 0.05 TESTFET On-Chain Settlement with direct public Block Explorer verification URLs!

Dimension	Traditional Human Platforms (HackerOne / Immunefi)	PatchBounty (@aegis Autonomous Swarm)
Average Cost per Critical Bug	$2,400 - $6,200+ USD (+25% platform fee)	$140 - $305 USD (~90% Savings, 0% Researcher Cut)
What Company Receives	Bug description report ONLY (No patch!)	Full Multi-Bug Fixed Code + Git .patch + PDFs + .ics
Turnaround Time	7 to 21 Days Manual Human Triage	< 5 Seconds End-to-End Automated Sandbox Proof
Pre-Payment Theft Protection	High risk — company sees line & fixes for free	Zero-Leak HTTP 402 Server Seal + Blue Blurred Vault
Pre-Payment Quality Proof	Manual trust / disputes common	Mandatory @specter + @warden Subprocess PASS Gate

Dimension	Traditional Human Platforms (HackerOne / Immunefi)	PatchBounty (@aegis Autonomous Swarm)
Clean Bug-Free Code Behavior	Noise / false low-severity spam reports	Honest 0 Bugs Found Verdict -> $0 Bounty Required
Payment Transparency	Opaque fiat / delayed bank wires	Instant Fetch.ai Dorado On-Chain TX + Live Explorer URL

PART 2 — END-TO-END WORKFLOW & SEQUENTIAL SECURITY GATES

2. How PatchBounty Works Step-by-Step (Zero Shortcuts, Zero Leaks)

A critical architectural upgrade in PatchBounty v5.0 is enforcing the true economic sequence of a real security marketplace so neither side can cheat or skip negotiation:

STEP 1 — Zero-Bias Multi-Language Full-File Scan (Page 1): Notice that Page 1 deliberately DOES NOT ask the user to type their maximum budget before scanning! Why? Because in an AI negotiation demo, if a user enters '$150' before scanning, a skeptical judge can ask: 'Did your researcher agents secretly read the $150 input variable to set their quote?' By scanning with zero budget input first, @recon inspects 100% of every file across 15 extensions (Python, JavaScript/Node.js, TypeScript, Solidity, Java, Go, PHP, C/C++, C#, Ruby, Rust, SQL). If the code is safe and clean, @recon immediately issues a Clean Security Clearance ($0 Bounty Needed). If vulnerabilities exist, ALL bugs across the entire file are extracted and numbered (#1, #2, #3...).

STEP 2 — Numbered Blind Audit Report + Blue Frosted-Blur Locked Solution Vault (Page 2): Every detected bug is shown as a numbered card with its Severity (CRITICAL / HIGH / MEDIUM), CVSS v3.1 score, plain-language Business Risk explanation, and cryptographic SHA-256 Sandbox Proof Hash — while exact file paths, exact line numbers, and fixed source code are strictly stripped on the backend server (serialize_scan_for_client(..., is_settled=False) + HTTP 402 on all download routes). Below the findings, the user sees the Blue Frosted-Blur Locked Vault Box rendered over server-masked diff placeholders (masked_diff_preview), so even inspecting the browser DOM in Chrome DevTools leaks 0% of the real fix! Crucially, Page 2 routes strictly forward to Step 3 (Negotiation & Sandbox Verification) — nobody can jump straight to payment and skip negotiation.

STEP 3 — Live Real-World Market Benchmark, Multi-Agent Bargaining & Mandatory Sandbox Gate (Page 3): First, PatchBounty queries live vulnerability telemetry from the NIST National Vulnerability Database REST API (services.nvd.nist.gov) alongside published HackerOne / Bugcrowd / Immunefi CWE benchmarks to display what human researchers charge ($1,800 - $6,200 USD) side-by-side against @blade, @kestrel, and @titan's starting quotes ($140 - $305 USD). Next, the Maintainer submits a live Counter-Offer ($ USD) in the Negotiation Console: @aegis bargains round-by-round with all 3 researcher agents down toward their Honest CVSS Severity Floor ($140 Critical, $90 High, $55 Medium). Finally, the user clicks 'Verify Selected Solution in Sandbox & Lock Deal' (/api/approve-solution), triggering @specter + @warden to execute real isolated subprocess checks confirming 100% exploit blocking and unit-test pass. Backend /api/confirm-bounty throws HTTP 400 unless BOTH deal_approved == True AND solution_verified_pass == True!

STEP 4 — Interactive IST Rollout Calendar, Live Fetch Dorado 0.05 TESTFET Settlement & External Explorer Verification (Page 4): The user selects from 10 upcoming calendar dates and 4 daily Asia/Kolkata (IST) deployment windows (with automatic real-time lockout of past windows today) and settles 0.05 TESTFET on Fetch.ai Dorado Testnet via @vault. Instantly upon block confirmation, the blur lifts across the entire app, exact line numbers and full fixed source code unlock, Printable Maintainer & Researcher PDFs (with 0% Maintainer budget leak) + .ics calendar invites unlock, AND three official clickable Fetch.ai verification links appear so anyone can independently verify the transaction on Fetch's own network!

Stage	Backend Endpoint	Security / State Gate Enforced	User-Visible Output
Step 1: Scan	POST /api/chat	Resets deal_approved=False & escrow_receipt=None	Clean $0 Pass OR Numbered Blind Bugs (#1..#N)
Step 2: Blind Audit	GET /api/state	HTTP 402 on .patch/source/PDFs; DOM mask active	Numbered Impact Cards + Blue Blurred Solution Box
Step 3A: Bargain	POST /api/negotiate-bounty	Enforces Honest CVSS Severity Floor per CWE	Live @blade/@kestrel/@titan Concession Transcript
Step 3B: Pass Gate	POST /api/approve-solution	Runs real Python / Node --check subprocess duel	Sets solution_verified_pass =True -> Unlocks Pay
Step 4A: Settle	POST /api/confirm-bounty	Broadcasts 0.05 TESTFET on dorado-1 via CosmPy	Unblurs Full Code + Exact Lines + PDFs + .ics
Step 4B: Verify	GET /api/verify-live-ledger/{tx}	Queries live rest-dorado.fetch.ai validator JSON	Code 0 Confirmation + Direct Fetch Explorer Links

PART 3 — DEEP DIVE: EVERY AGENT, ITS EXACT ROLE, PORT, PROTOCOL & AI MODEL

3. Complete Agent-by-Agent & AI Model Mapping Architecture

Hackathon judges ALWAYS ask two critical technical questions: 'Are these 6 agents real Fetch.ai uAgents or just prompt wrappers?' and 'Which exact AI model runs inside which agent, and why?' Below is the exact architectural truth implemented directly in backend/asi_one_agent.py, backend/config_store.py, backend/sandbox_verifier.py, and backend/dorado_escrow.py:

1. Six Real Registered Fetch.ai uAgents with Deterministic Cryptographic Identities: Every core agent (@aegis, @recon, @specter, @forge, @warden, @vault) is instantiated as a real uagents.Agent(...) with its own cryptographic Ed25519 seed, unique agent1... address on the Fetch network, dedicated port (8011 to 8016), and Windows Python 3.14+ event-loop safety via backend/__init__.py + Global Agentverse UTC Clock-Skew Healer.

2. True 50/50 Dual ASI:One LLM Load Balancer (asi1 + asi1-mini) on a Single API Key: Instead of hardcoding a generic third-party LLM, backend/config_store.py connects directly to Fetch.ai's native ASI:One Inference API (https://api.asi1.ai/v1) using OpenAI-compatible async clients protected by threading.BoundedSemaphore(2), SHA-256 prompt caching (180s TTL), and strict 50/50 round-robin alternation (_rr_turn % 2) between Fetch's deep reasoning model 'asi1' and high-speed model 'asi1-mini', with automatic 45-second per-model rate-limit failover so live demos never suffer HTTP 429 errors!

3. Hybrid Neuro-Symbolic Design (Why AI Models Never Hallucinate Line Numbers or Fake Sandbox Passes): A pure LLM alone can hallucinate wrong line numbers or claim a broken patch compiles. Therefore, PatchBounty pairs ASI:One neural reasoning (@aegis conversational memory, Hinglish/English bilingual Q&A, executive blind risk synthesis, and negotiation intelligence) with deterministic symbolic compilers & subprocess sandboxes (@recon Python AST + multi-language parser, @specter ephemeral subprocess exploit runner, @forge AST transformer & multi-language patch synthesizer, @warden bytecode + node --check regression gate, and @vault CosmPy Dorado blockchain ledger client).

Agent Handle	Port & Protocols	Engine / AI Model Used	Exact Technical Responsibility
@aegis (Coordinator)	Port 8012 | Chat v0.3.0 + Payment v0.1.0	ASI:One Dual Brain (50/50 asi1 + asi1-mini)	Master orchestrator, session memory QA (EN/Hinglish), live multi-agent bargain mediator & Agentverse Mailbox gateway
@recon (Scanner)	Port 8011 | GitHub REST + Multi-Lang Parser	Python ast.NodeVisitor + 15-Lang Sink Engine	Fetches live GitHub repos, scans 100% of every file across 15 languages, verifies clean code ($0), ranks #1..#N bugs
@specter (Red Team)	Port 8013 | Subprocess Harness	Isolated Python3 + Node v20 Subprocess Prover	Executes real attack payloads in ephemeral temp directories & generates zero-leak SHA-256 proof hashes

Agent Handle	Port & Protocols	Engine / AI Model Used	Exact Technical Responsibility
@forge (+ @blade/@kestrel/@titan)	Port 8014 | Patch & Diff Architect	AST Transformer + Multi-Lang Patcher + ASI:One	Synthesizes 3 full-file patches fixing ALL bugs, builds masked_diff_preview for Blurred Vault & executive summary
@warden (QA Gate)	Port 8015 | Zero-Regression Verifier	Python Bytecode + node --check + Unit Suite	Runs Unpatched vs Patched duel; physically locks Step 4 Payment until exploit_blocked & tests == PASS
@vault (Treasury)	Port 8016 | CosmPy Dorado-1 Ledger	CosmPy Aerial Client + Fetch Dorado LCD API	Enforces HTTP 402 code seal, broadcasts real 0.05 TESTFET on dorado-1, blocks replay TXs & generates Explorer URLs

PART 4 — DATA SOURCES, LIVE APIS, PRIVACY & BLOCKCHAIN VERIFICATION

4. How Every Piece of Live Data Enters, Flows & Persists Across PatchBounty

Judges deeply inspect where data comes from to check whether anything is mocked. In PatchBounty, EVERY data stream is 100% live, verifiable, and reproducible on any Windows or Linux laptop:

Data Stream A — Live Rate-Limit-Free GitHub & Multi-Language Source Code Ingestion (backend/github_scanner.py): When a user enters a GitHub URL (e.g., https://github.com/stamparm/DSVW for vulnerable multi-bug proof or https://github.com/jonschlinkert/is-odd for clean bug-free safe pass), @recon streams the live repository archive via https://github.com/{owner}/{repo}/archive/HEAD.zip (with zero 60 req/hr rate limits!) plus optional GITHUB_TOKEN support, inspecting 15 code extensions (.py, .js, .jsx, .ts, .tsx, .sol, .java, .go, .php, .c, .cpp, .rb, .cs, .rs, .sql). Clean bug-free code never locks anything and never asks for payment ($0 required).

Data Stream B — Live Global CVE & Industry Bounty Market Data (backend/market_benchmark.py): When vulnerabilities are identified, PatchBounty queries the official US Government NIST National Vulnerability Database REST API v2.0 (https://services.nvd.nist.gov/rest/json/cves/2.0?cweId=CWE-...) to retrieve live global CVE counts and sample real-world CVE identifiers for that exact CWE weakness, paired with published HackerOne / Bugcrowd / Immunefi CWE payout benchmarks.

Data Stream C — Live AI Reasoning, Persistent Session Memory & Agentverse Mailbox Bridge (backend/uagents_mesh.py + backend/asi_one_agent.py): Terminal 1 runs the FastAPI + React Web Studio (`python -m uvicorn backend.main:app --host 127.0.0.1 --port 8002`), while Terminal 2 runs the live Agentverse Mailbox connector (`python -m backend.uagents_mesh` or `connect_agentverse.bat`), printing the direct clickable Inspector URL (`https://agentverse.ai/inspect/?uri=http%3A//127.0.0.1%3A8012&address=agent1q...`) and mirroring live ASI:One conversations straight into the local dashboard via `/api/mirror-sync`.

Data Stream D — Live Fetch.ai Dorado Wallets, Before/After Balance Proof & External Companion Explorer Links (backend/dorado_escrow.py): Just like DealSetu, Page 4 displays live on-chain `TESTFET` balances for both the Maintainer Buyer Wallet (`fetch1...`) and PatchBounty Escrow Seller Wallet (`fetch1...`) with a 1-click `Sync / Faucet` button. Settling `0.05 TESTFET` (`50,000,000,000,000,000 atestfet`) shows live Before -> Deducted (`-0.0500 TESTFET`) -> After balance on the Buyer wallet, live Before -> Credited (`+0.0500 TESTFET`) -> After balance on the Seller wallet, and direct external verification URLs on `https://companion.fetch.ai/dorado-1/transactions/{TX_HASH}`, `https://companion.fetch.ai/dorado-1/accounts/{WALLET}`, `https://explore-dorado.fetch.ai/transactions/{TX_HASH}`, and `https://rest-dorado.fetch.ai/cosmos/tx/v1beta1/txs/{TX_HASH}`.

External System / Layer	Exact Live Endpoint / Command	How To Verify During Live Judge Demo
Terminal 1 — Web Studio	python -m uvicorn backend.main:app --port 8002	Opens http://127.0.0.1:8002 full 5-page Web Studio
Terminal 2 — Agentverse Bridge	python -m backend.uagents_mesh	Prints Inspector link -> Click Connect -> Mailbox on Agentverse

External System / Layer	Exact Live Endpoint / Command	How To Verify During Live Judge Demo
Clean Safe Repo ($0 / No Lock)	https://github.com/jonschlinkert/is-odd	Shows Green '0 Bugs Found ($0 Required)' — Zero Lock!
Vulnerable Repo (Multi-Bug Lock)	https://github.com/stamparm/DSVW	Finds 7 Bugs -> Blue Blurred Vault -> Negotiate -> Sandbox PASS
Fetch Companion Block Explorer	https://companion.fetch.ai/dorado-1/transactions/{TX}	Shows -0.05 FET Buyer & +0.05 FET Seller externally in browser

PART 4B — COMPLETE CODEBASE FILE MAP, REST API SPECIFICATION & STAGE DEMO SCRIPT

4B. Exact File-by-File Code Map & 3-Minute Stage Pitch Choreography

File / Endpoint	Layer	Exact Technical Implementation Inside Repository
backend/__init__.py	Python 3.14 Loop	ensure_event_loop() heals Python 3.14+ asyncio startup across Windows & Linux
backend/config_store.py	ASI:One Dual Brain	50/50 round-robin across asi1 & asi1-mini, 45s cooldown failover, .env writer
backend/asi_one_agent.py	6 Fetch uAgents	Registers @aegis/@recon/@specter/@forge/@warden/@vault + Chat & Payment Protocols
backend/github_scanner.py	Code Ingestion	GitHub REST Tree recursive fetcher across 15 extensions + inline code detector
backend/market_benchmark.py	Live Market API	Queries live NIST NVD CVE API v2.0 + HackerOne/Bugcrowd/Immunefi CWE medians
backend/sandbox_verifier.py	AST & Sandbox	Multi-lang analyzer, Clean $0 check, AST patcher, node --check & HTTP 402 seal
backend/dorado_escrow.py	Fetch Blockchain	CosmPy 0.05 TESTFET Dorado broadcast, Explorer URLs, Live Node check & Anti-Replay
backend/disclosure_artifacts.py	PDF & Calendar	10-Day IST Calendar (past-slot lock), Zero-Leak PDF 1.4 compiler & .ics generator
backend/qa_expert_router.py	Bilingual Memory	6-turn conversation memory, Hinglish/English detection & prompt-injection guard
POST /api/negotiate-bounty	Live Bargaining	Multi-agent counter-offer engine enforcing Honest CVSS Severity Floors
POST /api/approve-solution	Mandatory Gate	Runs Unpatched vs Patched duel; physically unlocks Step 4 Payment on PASS
GET /api/verify-live-ledger/{tx}	External Verify	Real-time query against https://rest-dorado.fetch.ai/cosmos/tx/v1beta1/txs/{tx}

3-Minute Judge Stage Demo Choreography: (0:00-0:35) Paste Clean Safe Python Code -> Show instant Green '$0 Bounty Needed — 0 Bugs Found' to prove zero fake alerts! (0:35-1:15) Paste https://github.com/stamparm/DSVW or 3-Bug Node.js API -> Show all bugs found in Numbered Points + Blue Frosted-Blur Locked Solution Vault (prove Inspect Element has zero leak!). (1:15-2:05) Open Page 3 -> Compare $2,650 Industry Market Average vs $185 AI Quote (92% cheaper!), negotiate live counter-offer ($155), and click 'Verify Solution & Lock Deal' to pass @warden sandbox tests. (2:05-3:00) Open Page 4 -> Pick IST rollout slot, pay 0.05 TESTFET live on Fetch Dorado Testnet, watch the blur vanish across all lines & code, and open Fetch Official Block Explorer live on screen!

PART 5 — 45 REAL HACKATHON JUDGE VIVA QUESTIONS & WINNING ANSWERS

5. Word-for-Word Defense Guide (Technical English + Quick Hinglish Pitch Points)

Q1. What real-world problem does PatchBounty solve in one crisp 30-second pitch?

Technical Answer: Traditional bug bounties take 1 to 3 weeks, cost $2,000 to $6,000+ USD per vulnerability, and only hand the company a PDF complaint — NOT working fixed code — while suffering from severe mutual distrust: if a researcher shows the fix first, the business can copy it for free; if the business pays first, the fix might not even compile! PatchBounty uses 6 autonomous Fetch.ai uAgents to scan multi-language repositories in seconds, prove exploitability inside isolated sandboxes while keeping the code fix blurred & HTTP-402 locked in escrow, negotiate bounty prices 90% below human market rates, verify 100% test pass via @warden, and settle on Fetch.ai Dorado Blockchain.

> ***Quick Hinglish Speaking Point:*** *Normal bug bounty me $2500+ lagte hain, hafton lagte hain, aur sirf report milti hai fix nahi. Aur agar researcher pehle code dikha de to company free me copy kar sakti hai. PatchBounty seconds me multi-language code check karke fix ko blur/lock rakhta hai, sandbox me PASS prove karta hai, bargain karwata hai, aur Dorado blockchain payment hote hi full code unlock karta hai.*

Q2. How do you prevent a company from seeing the bug location on your UI and fixing it themselves without paying the researcher?

Technical Answer: We enforce a strict two-layer server-side Zero-Leak Escrow Boundary (serialize_scan_for_client(..., is_settled=False) in backend/sandbox_verifier.py + _require_paid_escrow() in backend/main.py). Before 0.05 TESTFET confirms on Fetch Dorado Blockchain, the server NEVER sends file paths, line numbers, sink expressions, or real patch lines to the browser — it only sends business-level impact summaries, CVSS scores, cryptographic SHA-256 proof hashes, and masked placeholder lines for the Blue Blurred Vault box, while direct downloads (/api/artifacts/fixed-source, /api/artifacts/fix.patch) return HTTP 402 Payment Required. Even opening Chrome DevTools Inspect Element reveals zero real code before payment!

> ***Quick Hinglish Speaking Point:*** *Payment se pehle server browser ko exact file name, line number ya real code bhejta hi nahi hai — API level par HTTP 402 lock laga hai aur UI me sirf masked placeholder blur dikhta hai, isliye Inspect Element se bhi chori namumkin hai.*

Q3. Conversely, if the code is blurred before payment, how can a company trust that the researcher's fix actually works before paying?

Technical Answer: That is exactly why we built our Mandatory Step 3 Pre-Payment Sandbox Verification Gate (@specter + @warden via POST /api/approve-solution). Before Step 4 Payment is even allowed to unlock (enforced by HTTP 400 in POST /api/confirm-bounty), @warden independently compiles the researcher's proposed patch inside an isolated subprocess (Python AST/bytecode + Node.js v20 'node --check'), replays real attack payloads against both the unpatched code (proving EXPLOITED) and the patched code (proving EXPLOIT BLOCKED), and runs regression unit tests (100% PASS). The company pays ONLY after neutral agent @warden mathematically certifies the fix passes!

> ***Quick Hinglish Speaking Point:*** *Payment button tab tak lock rehta hai jab tak neutral agent @warden isolated sandbox me unpatched vs patched code chala kar prove nahi kar deta ki attack 100% block ho gaya aur code zero error ke saath compile ho raha hai.*

Q4. Earlier systems asked the company to type their maximum budget upfront — couldn't an AI agent cheat and read that budget? How did you solve this?

Technical Answer: Yes! Asking for a budget upfront creates 'budget peeking' suspicion. In PatchBounty v5.0, Step 1 scanning takes ZERO upfront budget input! Instead, @recon + market_benchmark.py calculate objective, transparent starting quotes based strictly on CWE severity, multi-bug count, live NIST NVD CVE data, and published HackerOne / Bugcrowd / Immunefi industry medians — pricing our AI agents ~88% to 92% below human market rates right out of the box. Only AFTER seeing both the real-world market average and the AI starting quotes does the company open Step 3 Live Bargaining to submit counter-offers!

> ***Quick Hinglish Speaking Point:*** *Hum scan ke time budget poochte hi nahi! Pehle AI internet benchmark (HackerOne/NIST NVD) dikhata hai aur khud ~90% sasta starting quote deta hai — uske baad Step 3 me user live counter-offer dekar bargain karta hai.*

Q5. Why couldn't a user just skip negotiation and click Pay immediately after seeing the bug report?

Technical Answer: We eliminated that shortcut at both the UI workflow layer AND the backend state machine! Page 2 (Audit Report) directs strictly to Step 3 (Negotiate & Verify Solution), and on the backend every fresh scan sets deal_approved = False and solution_verified_pass = False. Calling POST /api/confirm-bounty before completing Step 3 returns HTTP 400 ('Step 3 Required First'), guaranteeing price discovery, negotiation, and sandbox verification always execute cleanly.

> ***Quick Hinglish Speaking Point:*** *Humne UI aur Backend dono jagah hard state-lock lagaya hai — jab tak Step 3 me bid lock aur sandbox PASS nahi hota, Step 4 payment API HTTP 400 dekar block rakhti hai.*

Q6. What happens if someone pastes 100% safe, clean, bug-free code into PatchBounty?

Technical Answer: PatchBounty never invents fake bugs to extract money! When analyze_multilang_vulnerabilities() finds zero dangerous sinks across all lines of submitted code, it returns is_clean_code = True, total_bugs_found = 0, candidates = [], and phase = 'CLEAN_VERIFIED' — displaying a green 'Clean Security Clearance — 0 Vulnerabilities Detected ($0 Bounty Needed)' banner without generating fake low-severity alerts.

> ***Quick Hinglish Speaking Point:*** *Bug-free clean code daalne par hamara system koi fake bug nahi banata — seedha Green Clean Clearance aur '$0 Bounty Needed' dikhata hai.*

Q7. What happens during live bargaining if a company offers an absurdly low counter-offer like $10 USD for 3 Critical RCE/SQLi bugs?

Technical Answer: Every vulnerability severity tier has an Honest Minimum Severity Floor enforced by @aegis ($140 USD for Critical, $90 USD for High, $55 USD for Medium). If a maintainer offers $10 USD, the researcher agents refuse to drop below their honest floor, counter at their exact floor price (still saving the company ~$45+ off initial ask and >90% off market rate), and display a clear 'Below Honest Severity Floor' notice rather than delivering garbage unverified fixes.

> ***Quick Hinglish Speaking Point:*** *Agar koi $10 jaisa galat offer de, to agents apne Honest Severity Floor ($140 Critical) se neeche nahi girte aur reason ke saath honest minimum counter quote dete hain.*

Q8. Does PatchBounty charge a 20%-25% middleman commission from security researchers like HackerOne or Bugcrowd?

Technical Answer: Zero percent (0% commission) is deducted from the researcher's approved bounty payout! That zero-deduction guarantee is explicitly verified on the printable Researcher Payout Receipt PDF, encouraging top specialized security agents (@blade, @kestrel, @titan) to offer dramatically more competitive upfront prices to companies.

> ***Quick Hinglish Speaking Point:*** *Researcher se 0% platform commission liya jata hai — poora approved payout seedha researcher wallet me jata hai.*

Q9. How do you ensure the company's confidential counter-offer ceiling never leaks onto the Researcher's Payout Receipt PDF?

Technical Answer: In backend/disclosure_artifacts.py, generate_maintainer_clearance_pdf() and generate_researcher_payout_pdf() use strictly separated data projections. While the Maintainer Clearance Pass PDF records internal budget comparison metrics for internal CFO/CISO audit, generate_researcher_payout_pdf() never accesses or embeds maintainer_budget_usd — verified automatically by unit tests asserting b'$175 USD' not in res_pdf.content!

> ***Quick Hinglish Speaking Point:*** *Maintainer PDF aur Researcher PDF alag functions se bante hain — Researcher PDF me company ka internal budget field kabhi touch hi nahi hota (unit test se verified hai).*

Q10. Are @aegis, @recon, @specter, @forge, @warden, and @vault genuine Fetch.ai uAgents or just Python classes?

Technical Answer: All six are genuine instances of uagents.Agent from the official Fetch.ai uagents framework (defined in backend/asi_one_agent.py), each initialized with its own deterministic cryptographic seed, unique on-network 'agent1...' Bech32 address, dedicated port mapping (8011 through 8016), and real protocol manifests!

> ***Quick Hinglish Speaking Point:*** *Chhe ke chhe asli Fetch.ai uAgents hain — har ek ka apna cryptographic seed, asli 'agent1...' address, aur ■■■ port (8011-8016) hai.*

Q11. Which official Fetch.ai Agentverse protocols are registered and published on @aegis?

Technical Answer: Two official standard protocols from uagents_core.contrib.protocols are registered on @aegis with publish_manifest=True: (1) AgentChatProtocol v0.3.0 (handling StartSessionContent, TextContent, ChatAcknowledgement, and EndSessionContent for seamless ASI:One Chat discoverability) and (2) AgentPaymentProtocol v0.1.0 in role='seller' (handling RequestPayment, CommitPayment, CompletePayment, RejectPayment, and CancelPayment).

> ***Quick Hinglish Speaking Point:*** *@aegis par official AgentChatProtocol v0.3.0 aur AgentPaymentProtocol v0.1.0 dono publish_manifest=True ke saath registered hain.*

Q12. What are @blade, @kestrel, and @titan, and why are there 3 researcher tiers?

Technical Answer: Under @forge (our Patch Architect coordinator), @blade, @kestrel, and @titan represent three specialized autonomous security researcher tiers competing for the company's bounty: (1) @blade builds a Minimal Surgical Hotfix with lowest code churn and lowest price (30-Day Warranty), (2) @kestrel builds a Defense-in-Depth + Allowlist Guard (90-Day Warranty — Recommended), and (3) @titan builds a Full Enterprise Zero-Trust Architecture (180-Day SLA Guarantee). Real companies love choosing between fast hotfix vs deep hardening!

Quick Hinglish Speaking Point: *@blade fast surgical fix deta hai, @kestrel recommended defense-in-depth deta hai, aur @titan full enterprise zero-trust security deta hai — taaki company apni zaroorat aur budget ke hisaab se choose kar sake.*

Q13. Why did you include a 'Global Agentverse UTC Clock Skew Healer' inside backend/asi_one_agent.py?

Technical Answer: During real testing on Windows laptops, even a 3-to-10 second local system clock drift on Windows causes Agentverse Mailbox challenge signatures to fail with '401 Unauthorized'! At startup, asi_one_agent.py checks the HTTP Date header from https://agentverse.ai and automatically calibrates Python's time.time() offset if local drift exceeds 2 seconds — guaranteeing zero 401 errors on any judge's Windows machine!

Quick Hinglish Speaking Point: *Windows laptops ki ghadi agar 4-5 second aage-peeche ho to Agentverse Mailbox 401 error deta hai — hamara Clock Skew Healer startup par hi UTC match karke 401 ko 100% khatam kar deta hai.*

Q14. How did you fix Python 3.14+ asyncio RuntimeError crashes when importing uAgents & CosmPy on modern Windows systems?

Technical Answer: Python 3.14 removed implicit auto-creation of event loops in asyncio.get_event_loop() when called at import time by libraries like uagents and cosmpy. We engineered ensure_event_loop() inside backend/__init__.py and pass explicit loop=_AEGIS_BOOT_LOOP to every uAgent constructor so 'run.bat' boots cleanly across Python 3.10, 3.11, 3.12, 3.13, and 3.14+!

Quick Hinglish Speaking Point: *Python 3.14 me get_event_loop() crash hota tha — backend/__init__.py me ensure_event_loop() bana kar har Python version par smooth startup fix kiya gaya hai.*

Q15. How do Agentverse ASI:One live chats sync into the local Web Studio UI in real time?

Technical Answer: Whenever an external user or hackathon evaluator chats with @aegis on ASI:One / Agentverse via AgentChatProtocol, handle_asi_one_chat() posts the turn + live scan bundle to http://127.0.0.1:8002/api/mirror-sync, which persists it to data/session_store.json and broadcasts immediately over WebSockets (/ws) to all open browser tabs!

Quick Hinglish Speaking Point: *Jab koi ASI:One par @aegis se chat karta hai, /api/mirror-sync aur WebSocket ke zariye woh live conversation hamare Web UI par bhi real-time dikhti hai.*

Q16. Why do all 6 agents use single-word callsigns (@aegis, @recon, @specter, @forge, @warden, @vault)?

Technical Answer: Single-word hard callsigns make multi-agent telemetry logs, Agentverse handles, chat mentions, and architecture diagrams instantly scannable without verbose hyphenated slugs, while matching each agent's exact security role.

Quick Hinglish Speaking Point: *Single-word names (@aegis, @recon, @specter, @forge, @warden, @vault) clean, memorable aur real security operations logs me padhne me super clear hote hain.*

Q17. How does EndSessionContent work in AgentChatProtocol and why does it matter for automated evaluation?

Technical Answer: When a stateless policy/meta question is sent over AgentChatProtocol, returning [TextContent(...), EndSessionContent(type='end-session')] after sending ChatAcknowledgement tells ASI:One's orchestrator that the response turn is complete and prevents evaluation timeouts.

Quick Hinglish Speaking Point: *ChatAcknowledgement aur EndSessionContent lagane se ASI:One evaluator ko turant clean completion signal milta hai aur kabhi timeout nahi hota.*

Q18. Can users configure API keys directly from the Web UI without manually editing hidden .env files on Windows?

Technical Answer: Yes! Clicking 'Connect API Keys' in the top-right navbar from ANY page opens a live configuration modal where entering ASI_ONE_API_KEY, routing mode, optional GITHUB_TOKEN, AGENTVERSE_API_KEY, or DORADO_BUYER_SEED writes directly to local .env AND executes a live test call (POST /api/test-asi) against Fetch's ASI:One endpoint!

Quick Hinglish Speaking Point: *Haan! Top navbar ke 'Connect API Keys' button se seedha UI me key daal kar 'Save & Test Live API' click karo — apne aap .env me save aur live verify ho jata hai.*

Q19. Which exact LLM models run inside PatchBounty, and why didn't you just use OpenAI GPT or Gemini APIs?

Technical Answer: PatchBounty uses 100% native Fetch.ai ASI:One models — specifically 'asi1' (flagship deep reasoning model) and 'asi1-mini' (high-speed low-latency model) accessed via https://api.asi1.ai/v1 using a single ASI_ONE_API_KEY. Using Fetch's native LLM stack makes the entire intelligence layer 100% aligned with the Fetch.ai ecosystem.

Quick Hinglish Speaking Point: *Humne OpenAI/Gemini ke bajaye 100% Fetch.ai ke apne native models 'asi1' aur 'asi1-mini' (https://api.asi1.ai/v1) use kiye hain.*

Q20. How does your 50/50 Dual-Model Load Balancer work internally in backend/config_store.py?

Technical Answer: RuntimeConfig maintains a thread-safe counter (_rr_turn protected by threading.Lock()). Even turns route primary=['asi1', 'asi1-mini'] while odd turns route primary=['asi1-mini', 'asi1'], achieving a true 50/50 load split across both models on one API key. If either model hits a temporary rate limit, _model_cooldown_until isolates that model for 45 seconds and instantly fails over to its sibling model within the same request — giving 0% failed requests!

Quick Hinglish Speaking Point: *Har call par thread-safe counter se 50% calls 'asi1' aur 50% 'asi1-mini' par jati hain, aur agar ek model busy ho to 0 second me doosra model jawab de deta hai.*

Q21. How many programming languages can PatchBounty inspect, and does it stop at the first bug or scan the entire file?

Technical Answer: PatchBounty supports 15 file extensions covering 12+ languages: Python (.py), JavaScript/Node.js (.js, .jsx), TypeScript (.ts, .tsx), Solidity Smart Contracts (.sol), Java (.java), Go (.go), PHP (.php), C/C++ (.c, .cpp), Ruby (.rb), C# (.cs), Rust (.rs), and SQL (.sql). Unlike single-bug scanners, it inspects EVERY line from top to bottom across all fetched files and lists ALL bugs as numbered points (#1, #2, #3... e.g., finding all 7 real bugs in GitHub's stamparm/DSVW in one pass!).

Quick Hinglish Speaking Point: *Python, Node.js, TypeScript, Solidity, Java, Go, PHP, C/C++ samet 12+ languages support hoti hain — aur pehle bug par rukne ke bajaye poora code check karke saare bugs #1, #2, #3... points me batata hai.*

Q22. How does Python vulnerability detection differ from regex scanning, and why is Python AST transformation special?

Technical Answer: For Python code, @recon parses source into a real Abstract Syntax Tree (ast.parse) so comments or docstrings don't cause false positives, and @forge runs _AegisASTSecurityTransformer (an ast.NodeTransformer subclass) that structurally rewrites unsafe AST nodes (f-string SQL queries -> parameterized '?' tuples, shell=True -> shlex argv safe runner, eval() -> ast.literal_eval(), yaml.load() -> yaml.safe_load(), os.path.join() -> realpath/commonpath confinement, md5/sha1 -> sha256) and verifies ast.parse() bytecode compilation on the output!

> ***Quick Hinglish Speaking Point:*** *Python ke liye sirf regex nahi balki real Python AST compiler tree use hota hai, jo unsafe code ko safe AST tree me transform karke dobara compile check karta hai.*

Q23. How do you verify JavaScript / Node.js patches in the sandbox without risking host machine compromise?

Technical Answer: Inside an ephemeral temporary directory (tempfile.TemporaryDirectory()), @warden writes the remediated JavaScript file and runs Node.js v20's compiler check ('node --check file.js') with a strict 4-second subprocess timeout, confirming zero syntax errors while static & dynamic sink verification confirms every SQL concatenation, child_process.exec RCE, and hardcoded Stripe/AWS key has been neutralized.

> ***Quick Hinglish Speaking Point:*** *Node.js patches ko isolated temp folder me asli 'node --check' compiler aur sink verifier se guzara jata hai taaki broken code kabhi pass na ho.*

Q24. How do you detect and fix Solidity Web3 smart-contract vulnerabilities like Reentrancy and tx.origin?

Technical Answer: In Solidity (.sol), @recon detects external ETH transfers (.call{value: ...}('')) occurring before state balance updates (CWE-841 Classic DAO Reentrancy, CVSS 9.8) as well as dangerous phishing-vulnerable authentication checks (require(tx.origin == owner), CWE-346). @forge rewrites tx.origin to msg.sender and neutralizes raw call-value reentrancy exposure!

> ***Quick Hinglish Speaking Point:*** *Solidity smart contracts me DAO Reentrancy (.call{value:...}) aur tx.origin auth bug dono pakde aur fix kiye jate hain.*

Q25. Why does left-sidebar Chat support both English and Hindi/Hinglish seamlessly?

Technical Answer: In backend/qa_expert_router.py, _looks_hinglish() detects Roman Hindi/Hinglish tokens ('kya', 'kyu', 'kaise', 'paisa', 'mera', 'kitne', etc.) and instructs ASI:One to answer naturally in Roman Hinglish while injecting the last 6 conversation turns + live session state (respecting zero-leak rules before payment!). If asked in English, it answers in crisp executive English.

> ***Quick Hinglish Speaking Point:*** *qa_expert_router.py automatically pehchan leta hai ki user Hinglish me pooch raha hai ya English me, aur pichle messages + current scan context yaad rakhkar usi language me jawab deta hai.*

Q26. How did you stop the AI chat assistant itself from accidentally revealing unpaid line numbers if a user tries prompt injection in chat?

Technical Answer: Architectural isolation! Look at _build_safe_live_context() in backend/qa_expert_router.py: when phase != 'SETTLED', we DO NOT EVEN PASS the line numbers or fixed source code into the LLM's system prompt! Even a 100% genius prompt-injection attack cannot extract secret line numbers or patch code from an LLM whose context window literally does not contain them!

Quick Hinglish Speaking Point: *Sabse strong security trick — payment se pehle LLM ke prompt me hum line numbers aur fixed code bhejte hi nahi hain! Jab AI ke paas woh data hai hi nahi, to prompt injection se bhi leak hona 100% impossible hai!*

Q27. Why were stars (**bold**) showing in chat earlier, and how did you fix chat typography?

Technical Answer: Plain text divs print raw Markdown characters like '**bold**' and '&'. We built a dedicated React rich-text component (<FormattedMessage /> in App.jsx) plus html.unescape() on the backend that cleanly converts **bold** into semantic tags, `inline code` into styled pills, bullet lists into clean rows, and URLs into clickable external links!

Quick Hinglish Speaking Point: *React me custom <FormattedMessage /> parser banaya gaya hai jo stars hata kar real bold text, code badges aur clickable links dikhata hai.*

Q28. What happens if internet access drops or ASI:One API key hasn't been entered yet during local testing?

Technical Answer: Every single module in PatchBounty has graceful hybrid resilience: AST/multi-language scanning, isolated sandbox execution, market CWE benchmarks, bargaining math, PDF/ICS generation, and Hinglish/English smart fallback answers work 100% deterministically in < 200ms even before entering an API key — and the instant ASI_ONE_API_KEY is connected, live dual-model neural reasoning kicks in automatically!

Quick Hinglish Speaking Point: *Bina API key ya slow internet par bhi app 0% crash ke saath chalti hai, aur API key lagate hi live asi1 + asi1-mini active ho jate hain.*

Q29. Where do your Real-World Industry Market Bounty numbers come from — are they random numbers?

Technical Answer: No! backend/market_benchmark.py combines two verifiable sources: (1) Live HTTP queries to the official US NIST National Vulnerability Database API v2.0 (https://services.nvd.nist.gov/rest/json/cves/2.0?cweId=...) returning real-time global CVE counts and sample CVE IDs for the exact CWE found in your code, and (2) Published 2025-2026 CWE payout medians across HackerOne, Bugcrowd, and Immunefi scaled dynamically by multi-bug count.

Quick Hinglish Speaking Point: *Bilkul real data hai — live US Government NIST NVD API se global CVE count aata hai aur HackerOne/Bugcrowd/Immunefi ke CWE benchmarks se real comparison hota hai.*

Q30. How does live GitHub repository scanning work under the hood?

Technical Answer: backend/github_scanner.py extracts owner/repo from any GitHub URL, queries GitHub's recursive git tree API (/repos/{owner}/{repo}/git/trees/HEAD?recursive=1), prioritizes security-relevant application source files across 15 extensions while skipping test/vendor folders, fetches live raw file contents via raw.githubusercontent.com, and passes all files into @recon.

Quick Hinglish Speaking Point: *GitHub REST Tree API se poori repo file list nikal kar live raw code fetch hota hai aur saari code files top-to-bottom check hoti hain.*

Q31. Does PatchBounty support scanning private corporate GitHub repositories too?

Technical Answer: Yes! Opening 'Connect API Keys' in the top navbar and pasting a GitHub Personal Access Token (GITHUB_TOKEN) automatically authenticates all GitHub API and raw source requests with 'Authorization: Bearer <token>', allowing private enterprise repos to be audited seamlessly.

Quick Hinglish Speaking Point: *Haan! Top navbar ke 'Connect API Keys' me apna GITHUB_TOKEN daalte hi private corporate GitHub repos bhi scan ho jati hain.*

Q32. Why does a bug bounty platform need an Interactive Deployment Rollout Calendar (Page 4)?

Technical Answer: In production engineering, companies don't blindly deploy security patches during peak daytime traffic! And researchers follow Responsible Coordinated Disclosure windows (agreeing NOT to publish their CVE writeup before the company merges the patch in production). Our 10-Day Asia/Kolkata (IST) Calendar lets both sides lock an exact deployment date & time slot (Morning CI, Afternoon Canary, Night Low-Traffic, Midnight Window) and downloads separate .ics calendar invites for Maintainer and Researcher!

> ***Quick Hinglish Speaking Point:*** *Real companies peak traffic me patch deploy nahi karti aur researcher ko bhi disclosure date chahiye hoti hai — isliye IST Calendar dono ke liye exact rollout time aur .ics calendar invite banata hai.*

Q33. What happens on the Calendar if it is currently 9:00 PM IST today — can someone accidentally book 10:00 AM IST today?

Technical Answer: No! In backend/disclosure_artifacts.py, build_seven_day_disclosure_slots() computes real-time Asia/Kolkata (UTC+05:30) timestamps and automatically marks any today time window whose start time is within 15 minutes or already past as available = False ('Closed Today'), disabling it in the UI AND blocking it with HTTP 400 on POST /api/select-slot!

> ***Quick Hinglish Speaking Point:*** *Real-time IST clock check laga hai — aaj ka jo time nikal chuka hai woh UI aur Backend dono jagah 'Closed Today' hokar lock ho jata hai.*

Q34. How do you generate PDF certificates without requiring heavy OS binaries like wkhtmltopdf or LaTeX on Windows?

Technical Answer: backend/disclosure_artifacts.py includes a pure zero-dependency native PDF 1.4 vector compiler (_compile_single_page_vector_pdf) written in pure Python bytes! It compiles crisp vector headers, tables, Merge PINs, SHA-256 digests, and Dorado TX hashes in < 3 milliseconds on ANY Windows/Mac/Linux machine with zero external binary dependencies!

> ***Quick Hinglish Speaking Point:*** *Humne pure Python me custom PDF 1.4 vector generator likha hai — isliye Windows par bina kisi extra software install kiye 3 millisecond me crisp PDF banta hai.*

Q35. How does chat & scan history stay saved across browser refreshes and server restarts?

Technical Answer: Every state transition writes atomically to project-2-patchbounty/data/session_store.json via _save_state_to_disk(), and _load_persisted_state() restores full chat history, scan findings, negotiation history, and payment status automatically on startup until the user clicks 'New Chat'.

> ***Quick Hinglish Speaking Point:*** *Har chat aur scan action turant disk par data/session_store.json me save hota hai — refresh ya server restart karne par bhi kuch gayab nahi hota.*

Q36. Are any API keys or private wallet seeds ever leaked inside the project .zip submission?

Technical Answer: Strictly zero! Our packaging pipeline explicitly excludes .env, session_store.json, and runtime_keys.json, verifies zero secret substrings exist anywhere in the archive, and enforces 100% unique case-insensitive filenames (< 2 MB clean archive) for clean Windows extraction.

> ***Quick Hinglish Speaking Point:*** *Zip file me .env aur keys 100% excluded hain aur Windows case-collision check bhi pass hai.*

Q37. Is your 0.05 TESTFET blockchain transaction simulated, or is it a real live transaction on Fetch.ai's blockchain?

Technical Answer: It is 100% LIVE on Fetch.ai's official Dorado Testnet (chain_id: 'dorado-1')! In backend/dorado_escrow.py, LedgerClient(NetworkConfig.fetchai_stable_testnet()) signs and broadcasts a real Cosmos MsgSend transfer of 50,000,000,000,000,000 atestfet (0.05 TESTFET) from the Maintainer uAgent wallet to the Researcher uAgent wallet, calls tx.wait_to_complete(), and returns the real 64-character hexadecimal transaction hash and confirmed block height!

> ***Quick Hinglish Speaking Point:*** *100% REAL on-chain transaction hai! CosmPy ke zariye Fetch.ai Dorado Testnet par asli 0.05 TESTFET transfer hota hai aur real Block Height + 64-char TX Hash milta hai.*

Q38. How can a researcher or hackathon judge verify on Fetch.ai's OWN external software that payment really happened?

Technical Answer: Right inside the Receipt card on Page 4 (and in Chat), PatchBounty provides FOUR direct external Fetch.ai verification links + a live validator check button: (1) Official Fetch Block Explorer UI: https://explore-dorado.fetch.ai/transactions/{TX_HASH}, (2) Fetch Companion Explorer: https://companion.fetch.ai/dorado-1/transactions/{TX_HASH}, (3) Official Fetch Validator Raw Ledger JSON: https://rest-dorado.fetch.ai/cosmos/tx/v1beta1/txs/{TX_HASH}, and (4) Researcher Wallet Account Ledger: https://explore-dorado.fetch.ai/account/{WALLET}. Clicking 'Copy Verify URL' lets you share the link with anyone in the world to verify independently!

> ***Quick Hinglish Speaking Point:*** *Payment hote hi Fetch.ai ke official Block Explorer aur Validator Node ke direct clickable links milte hain — researcher ya judge apne phone/laptop browser me khol kar verify kar sakta hai ki 0.05 TESTFET wallet me aa chuka hai!*

Q39. What does the 'Re-Verify Live on Fetch Node' button do behind the scenes?

Technical Answer: Clicking 'Re-Verify Live on Fetch Node' calls GET /api/verify-live-ledger/{tx_hash}, which queries Fetch.ai's live validator node (https://rest-dorado.fetch.ai/cosmos/tx/v1beta1/txs/{tx_hash}) in real time and parses code == 0 (SUCCESS), confirmed block height, UTC block timestamp, exact atestfet transferred, and sender/recipient wallet addresses!

> ***Quick Hinglish Speaking Point:*** *Yeh button seedha Fetch.ai ke live validator server se real-time status pooch kar Validator Code = 0 (SUCCESS), Block Height aur Timestamp screen par dikhata hai.*

Q40. What happens if a dishonest user copies an old confirmed TX hash from yesterday and tries to reuse it to unlock another repository's fix for free?

Technical Answer: Blocked instantly by @vault's Anti-Replay Guard! _SEEN_TX_HASHES records every consumed Dorado transaction hash. Submitting any already-used TX hash to POST /api/verify-tx-hash immediately triggers status = 'REPLAY_BLOCKED', issues an AgentPaymentProtocol CancelPayment event, deducts 0 FET, and keeps the code vault strictly locked!

> ***Quick Hinglish Speaking Point:*** *Purana TX hash dobara daalne par @vault ka Anti-Replay Guard turant 'REPLAY_BLOCKED' kar deta hai aur code lock hi rehta hai!*

Q41. What happens if someone submits a fake or random 64-character hex string as a TX hash?

Technical Answer: @vault queries https://rest-dorado.fetch.ai/cosmos/tx/v1beta1/txs/{tx_hash}. When the validator node returns non-200 or tx_response.code != 0, @vault rejects it immediately with status = 'PAYMENT_CANCELLED' and keeps all fixes HTTP-402 sealed!

Quick Hinglish Speaking Point: *Fake hex hash daalne par Fetch validator node par TX nahi milta aur @vault turant reject karke vault lock rakhta hai.*

Q42. What if the Maintainer wallet runs low on TESTFET during back-to-back judge demos?

Technical Answer: In backend/dorado_escrow.py, before broadcasting each transaction, _sync_broadcast_dorado_escrow() checks the live bank balance in atestfet; if it falls below 4x the escrow requirement (0.20 TESTFET), it automatically invokes uAgents' official fund_agent_if_low() faucet helper to top up the wallet seamlessly!

Quick Hinglish Speaking Point: *Agar demo ke dauran TESTFET kam ho jaye, to code apne aap Fetch faucet (fund_agent_if_low) call karke wallet top-up kar leta hai!*

Q43. What does the 'CancelPayment (0 FET)' button demonstrate for Fetch.ai protocol compliance?

Technical Answer: Full AgentPaymentProtocol v0.1.0 lifecycle support requires handling aborted/rejected payments gracefully, not just happy-path payments. Clicking CancelPayment emits a formal CancelPayment receipt (0 TESTFET charged), keeps the researcher's patch safely locked inside @vault, and allows the maintainer to resume settlement anytime later.

Quick Hinglish Speaking Point: *Yeh dikhata hai ki agar deal cancel ho to 0 FET kat-ta hai aur researcher ka code bhi safe & locked rehta hai.*

Q44. How could PatchBounty scale from Dorado Testnet to Fetch Mainnet & enterprise CI/CD pipelines after the hackathon?

Technical Answer: Two clean config switches take PatchBounty to full production: (1) Switching NetworkConfig.fetchai_stable_testnet() to NetworkConfig.fetchai_mainnet() settles real FET/USDC bounties globally in seconds, and (2) Triggering POST /api/chat from a GitHub Action on every Pull Request automatically audits new code before it ever reaches production!

Quick Hinglish Speaking Point: *Sirf NetworkConfig ko mainnet par switch karke aur GitHub Actions PR webhook jod kar ise kal hi real SaaS product banaya ja sakta hai.*

Q45. How did team members Atul Anand, Jeeya, and Vanshika divide and integrate the system architecture?

Technical Answer: The architecture cleanly spans three tightly integrated pillars built by Atul Anand, Jeeya, and Vanshika: (1) Core Fetch.ai 6-uAgent Mesh, AgentChat/Payment Protocols, & Dorado CosmPy Escrow (@aegis, @vault), (2) Multi-Language AST/Compiler Vulnerability Engine, Isolated Subprocess Sandbox Gate, & Live NIST Market Benchmarks (@recon, @specter, @forge, @warden), and (3) Interactive Multi-Page Daylight Studio UI, Blurred Vault Lock UX, Live Bargaining Console, IST Calendar, & PDF/ICS Artifact Engine.

Quick Hinglish Speaking Point: *Team (Atul Anand, Jeeya, aur Vanshika) ne milkar 3 pillars banaye: (1) Fetch 6-uAgents + Dorado Blockchain Escrow, (2) Multi-Language Scanner + Sandbox Verifier + Market Benchmark, aur (3) Clean Daylight UI + Blurred Lock + Calendar/PDF System.*

Q46. Why is using BOTH AST symbolic compilation AND ASI:One neural reasoning stronger than using a pure LLM agent?

Technical Answer: A pure LLM alone can hallucinate line numbers or output syntactically broken patches that crash production servers. By combining Python AST / Node v20 compiler checks + ephemeral subprocess exploit testing (@recon, @specter, @forge, @warden) WITH Fetch.ai ASI:One neural reasoning (@aegis), PatchBounty achieves 0% hallucinated line numbers AND 100% syntactically verified patches while still providing rich bilingual conversational intelligence!

> ***Quick Hinglish Speaking Point:*** *Sirf LLM use karne se AI galat line number ya toota hua code de sakta hai — humne Real Compiler/AST + ASI:One dono mila diye jisse 0% hallucination aur 100% working fix milta hai.*

Q47. What prevents malicious scanned code inside the sandbox from deleting host files or hanging your backend server forever?

Technical Answer: Three isolation controls protect the host server inside backend/sandbox_verifier.py: (1) Code executes in a separate child process inside an isolated ephemeral directory (tempfile.TemporaryDirectory()) that is wiped immediately on exit, (2) Every subprocess enforces a hard wall-clock timeout (timeout=4.0 seconds) so infinite loops terminate cleanly, and (3) Static compilation & controlled mock harnesses run rather than executing arbitrary internet-facing scripts.

> ***Quick Hinglish Speaking Point:*** *Har test alag child process aur temporary folder me hard 4-second timeout ke saath chalta hai aur khatam hote hi temp folder delete ho jata hai.*

Q48. Can a maintainer switch between @blade, @kestrel, and @titan AFTER negotiating a counter-offer?

Technical Answer: Yes! When POST /api/negotiate-bounty runs, all three researcher agents (@blade, @kestrel, and @titan) update their negotiated prices simultaneously based on their individual honest floors. The maintainer can click between any of the three bids on Page 3, re-verify that specific patch in the sandbox via @warden, and proceed to Step 4 settlement with whichever tier best fits their SLA requirements!

> ***Quick Hinglish Speaking Point:*** *Bilkul! Negotiation ke baad teeno agents ke naye rate dikhte hain aur aap @blade, @kestrel, ya @titan kisi ko bhi choose karke sandbox pass karwa sakte hain.*

Q49. Why do both Maintainer and Researcher receive separate .ics Calendar files in addition to separate PDF certificates?

Technical Answer: In real enterprise DevSecOps, security hotfixes must be merged during coordinated maintenance windows while researchers schedule their public advisory release right after patch rollout. Downloading standard RFC-5545 .ics calendar files imports directly into Google Calendar, Microsoft Outlook, and Apple Calendar with exact Asia/Kolkata (IST) timestamps and Dorado TX proof!

> ***Quick Hinglish Speaking Point:*** *.ics calendar file Google Calendar aur Outlook me 1-click me add ho jati hai taaki deployment aur disclosure window kabhi miss na ho.*

Q50. If you had 30 more days post-hackathon, what 3 features would you ship next for global enterprise launch?

Technical Answer: 1. Automated GitHub Pull Request Bot: After Dorado settlement confirms on-chain, @forge opens a signed private GitHub PR directly on the company's branch. 2. Formal Solidity Symbolic Execution (Slither/Foundry integration inside @specter) for DeFi TVL pools. 3. Cross-Chain USDC + Mainnet FET Enterprise Treasury Pool on @vault so Web2 CFOs can fund bounties via Stripe/USDC while researchers get instant FET/USDC settlement.

> ***Quick Hinglish Speaking Point:*** *Agle 30 din ka roadmap: (1) Paid hote hi automatic GitHub Pull Request open karna, (2) DeFi smart contracts ke liye Foundry symbolic prover, aur (3) Mainnet FET + USDC payment pool.*

PART 6 — RAPID TEAM REVISION CHEAT SHEET (TECHNICAL TERMS IN SIMPLE WORDS)

6. Instant Glossary for Stage Pitch & Live Judge Q&A;

Technical Term	1-Line Plain Meaning to Speak Confidently in Viva
Fetch.ai uAgents	Autonomous microservice agents with unique 'agent1...' cryptographic identities & ports.
AgentChatProtocol v0.3.0	Fetch standard chat protocol enabling live discovery & messaging on ASI:One / Agentverse.
AgentPaymentProtocol v0.1.0	Fetch standard escrow payment protocol (RequestPayment, CommitPayment, CompletePayment, CancelPayment).
ASI:One (asi1 + asi1-mini)	Fetch.ai's native LLMs — we split traffic 50/50 across deep reasoning (asi1) & low latency (asi1-mini).
CWE (Common Weakness Enum.)	Global ID code for bug categories (e.g., CWE-89 = SQL Injection, CWE-841 = Solidity Reentrancy).
CVSS Score (0.0 - 10.0)	Industry severity score (9.0+ = CRITICAL, 7.0-8.9 = HIGH).
AST (Abstract Syntax Tree)	Compiler tree structure of Python code used by @recon/@forge for 100% accurate bug rewrite.
HTTP 402 Payment Required	Server-level lock ensuring zero fixed source or .patch download before Dorado blockchain settlement.
Blue Blurred Vault Preview	UI blurred preview backed by server masked placeholders so even Inspect Element leaks 0% code.
atestfet / Dorado-1	Fetch.ai Testnet smallest unit: 50,000,000,000,000,000 atestfet = 0.05 TESTFET settled via CosmPy.
Anti-Replay Guard	Blocks reuse of old confirmed TX hashes so one payment can never unlock two separate bounties.